

Nano4 GP1 生醫 HPC 課程

第一堂：連線與 AI Agent

第 01 章 登入、2FA 與環境管理

第 02 章 Antigravity Remote-SSH 與三個 AI Agent

📖 線上講義：<https://gemini960114.github.io/Nano4-Docs/>

- 國家高速網路與計算中心研究員
- 具生物資訊學背景專長涵蓋高效能運算（HPC）、計算神經科學與生成式AI。
- 深耕果蠅腦神經連接體學，開發3D神經影像重建與分析技術，相關成果發表於 Current Biology、Cell Reports 等期刊。



關於國網中心

■ 關於國網中心

國研院國網中心於1991年成立

為我國唯一大型共用計算平台及高等學研專用寬頻網路之服務提供者
並深耕高效能計算、雲端虛擬化、網路資安、AI、資料分析等核心技術
為我國之科研能量奠基

國網中心積極配合國家科技政策

以超級電腦與寬頻網路為基石，
提供可信賴之巨量資料與AI雲端服務
為各界打造安全與便利之數位資源

From Enabler to
Enhancer

超級電腦是研發的火種，
幫助台灣各界研發團隊點燃最亮的成果！



國網HPC 叢集系統

台灣杉二號
TAIWANIA 2



台灣杉二號
(TAIWANIA 2)

- 2018/11
- 計算力 9 PFLOPS
- Top500 # 20
- Green500 # 10

台灣杉三號
TAIWANIA 3



台灣杉三號
(TAIWANIA 3)

- 2020/11
- 計算力 2.7 PFLOPS
- Top500 # 181
- Green500 # 69

Forerunner1
創進一號



創進一號
(Forerunner 1)

- 2023/11
- 計算力 3.5 PFLOPS
- Top500 # 221
- Green500 # 91

NANO



Nano 5

- 2025/06
- 13.06 PFLOPS
- Top500 # 118
- Green500 # 72

國網晶創26 Nano4系統

登上500大超級電腦第29名

1,760張 H200

144張 GB200



臺灣學術網路中樞

NCHC Dark Fiber Network Backbone

Resilience Enhancement of cyberinfrastructure in Taiwan

Dark Fiber Built

Extensions in 2025



生醫服務入口平台

<https://lions.nchc.org.tw/>



LIONS 國網生科雲

Life science Informatics ON-cloud Services

生醫計算核心設施服務

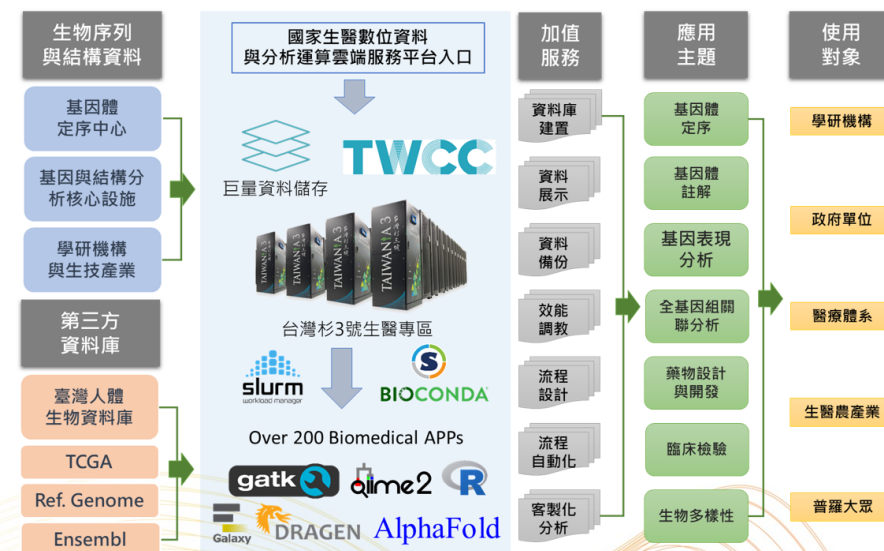


國家實驗研究院
NATIONAL INSTITUTES OF APPLIED RESEARCH



NCFB

生技醫藥核心設施平台
National Core Facility for Biopharmaceuticals



建構可信賴的生物醫學數據研究環境

- 以叢集式高效能或雲端計算主機為基礎，提供生命科學、醫學與農業領域分析計算時所需之函式庫、分析軟體、套裝程式等。
- 根據生物計算類之特性，將計算平台效能調教至最佳化，提供計算服務。
- 可提供大尺度之基因體分析計算，全基因體之間比對、關聯性分析，找尋生物標記、特定影像樣式，提供遺傳背景，以找尋疾病成因，解碼生命藍圖。

生成式AI應用開發平台

<https://rap.genai.nchc.org.tw/>

TAIWAN AI RAP 高效能AI應用開發平台

TAIWAN AI RAP — 快速啟動你的 AI 應用！

實現你的好點子，TAIWAN AI RAP 是你的 AI 神隊友！

由國研院國網中心所打造，專為加速台灣生成式 AI 的開發與應用而生。我們整合強大算力、實用 AI 工具及模型、模型微調與評估，提供高效能AI應用開發平台，幫助企業、團隊和研究人員快速上手、少走彎路，縮短開發時間，讓 AI 落實在各行各業中，創造實際價值。

三大特色功能

客製化流程
設計前台
Customized
Workflow Design
Frontend

多模型API
服務
Multi-Model
API Service

模型微調
與評估
Model Fine-Tuning
And Evaluation

核心願景

打造臺灣
AI聚落
Fostering an AI
Ecosystem in
Taiwan

第 0 章 準備

課前準備確認 🙋

#	項目	完成了嗎？
1	iService 帳號已加入計畫 <code>G0V115088</code>	☐
2	手機已安裝 IDExpert App 並完成 2FA 綁定	☐
3	已用 <code>ssh</code> 登入 Nano4 一次 (系統才會建立家目錄)	☐
4	已安裝 Google Antigravity 並用 Google 帳號登入	☐
5	已下載 <code>ssh-proxy</code> 執行檔	☐
6	(選做) ChatGPT 帳號 (Codex) 或 Claude 帳號 (Claude Code)	☐

還沒完成的同學請舉手，助教會協助你。

本課程的固定設定（請記住）

計畫代號：GOV115088（國網生技醫藥高效能運算推廣與應用計畫）

佇列：ngs62g

規格：每個作業固定 -c 8 --mem=62G（國網官方規定，不可自行縮減）

環境：CPU-only，本次不使用 GPU

講義裡出現的 H200 / GB200 GPU、GOV113021、MST109178 等都是參考資料，照抄會被排程器拒絕。

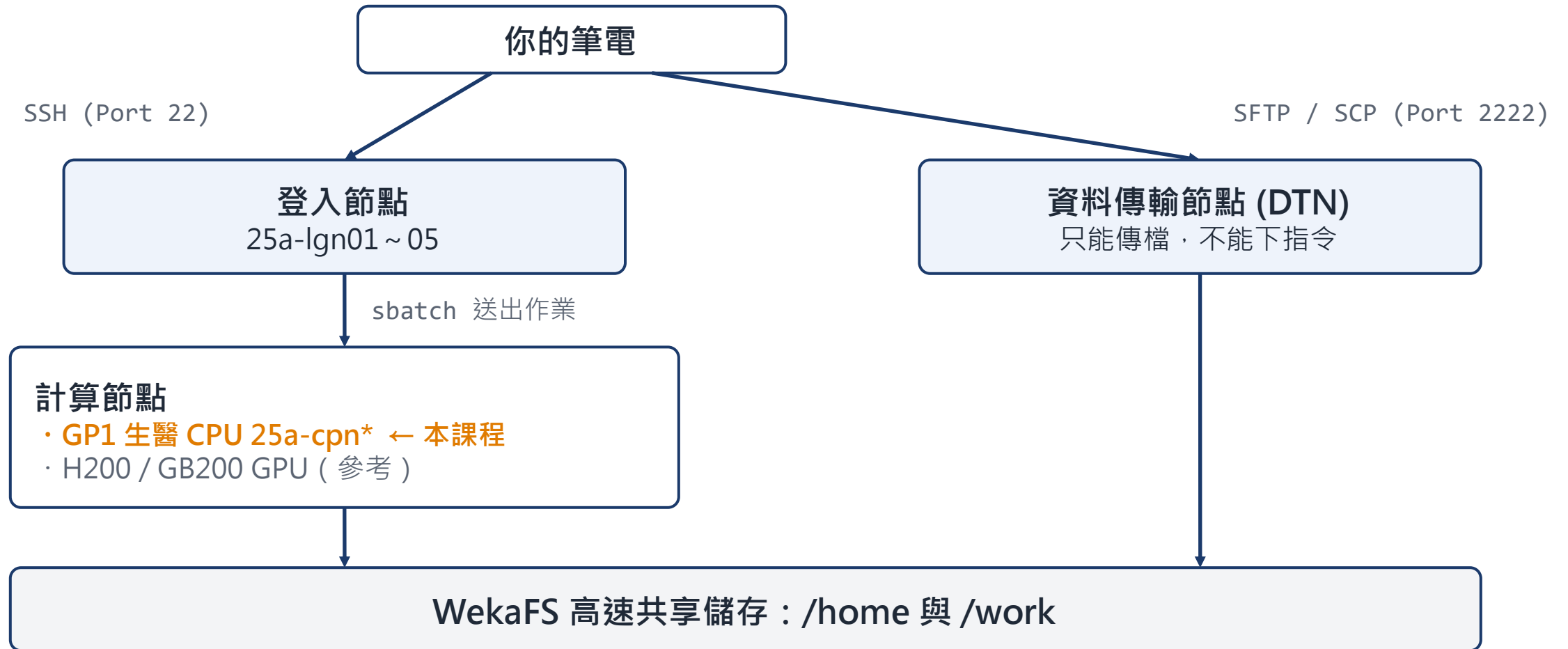
第 01 章

登入、2FA 與環境管理

第 01 章：13 個小節與 6 個練習

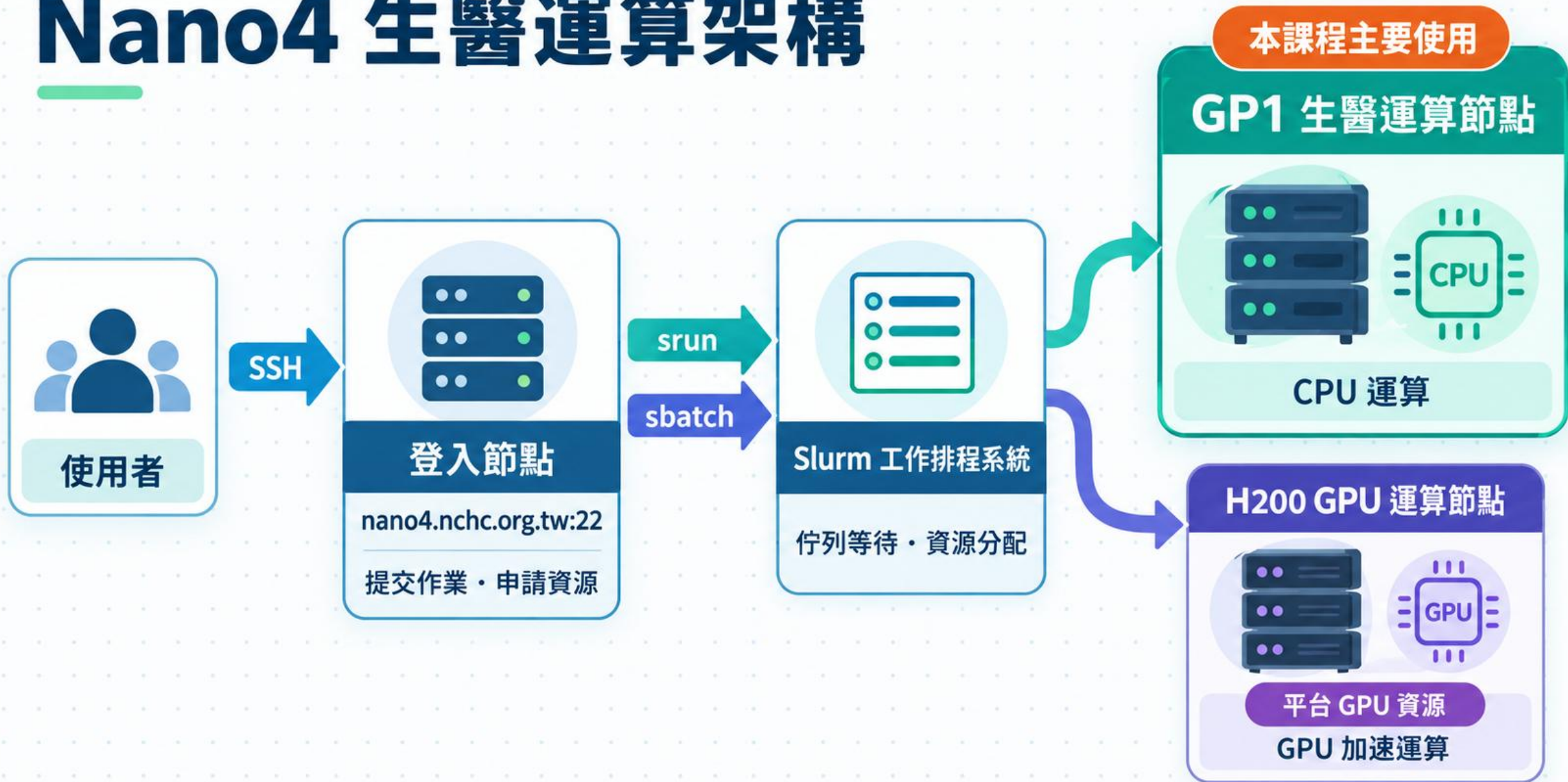
§	主題	本堂課
1	叢集前門：登入節點 SSH:22 vs 傳輸節點 SFTP:2222	講解
2	前置準備：iService 計畫與 IDExpert 2FA	課前完成
3	SSH 登入、三種 2FA、取得課程教材	實作
4	設定 SSH Config	實作
5	從台灣杉三號 (T3) 搬遷資料	參考
6	環境健檢與儲存空間 → 🧪 練習 1	實作
7	資料傳輸節點 DTN (Port 2222) → 🧪 練習 2	選做
8	Lmod 模組 module → 🧪 練習 3	實作
9	Apptainer 容器	第三堂實作
10	uv Python 環境 → 🧪 練習 4	選做
11	Slurm 佇列 ngs62g → 🧪 練習 5、6	時間夠再做
12	六個入門練習回顧	—
13	常見問題 FAQ	參考

§1 叢集前門：Nano4 長什麼樣子？



GP1 生醫節點是 Nano4 的一部分，和 GPU 節點共用登入節點與儲存空間。

Nano4 生醫運算架構



晶創26 Nano4 × GP1 生醫運算架構

運算驅動生醫 • 連結未來醫療
從資料到發現 • 從計算到健康

生醫研究者

透過網路連線
使用 Nano4 服務

HTTPS / 網路連線



nano4.nchc.org.tw

進入晶創26 (Nano4) 使用 GP1 生醫運算服務的統一入口

SSH · Port 22

SFTP / SCP · Port 2222



登入節點
25a-lgn01-05
提供指令操作、工作提交等服務



資料傳輸節點
DTN
提供檔案上傳 / 下載傳輸服務



GP1 生醫核心設施 | 本課程主軸

面向生醫研究的高效能運算資源，支援大規模資料分析與 AI 運算

Slurm



CPU 節點 ×18
128 cores · 1000G / 節點
適用於一般計算、資料分析



H200 GPU 節點 ×2
8 × H200 · 1600G / 節點
支援生成式 AI 與生醫深度學習
加速運算



大記憶體節點 ×2
128 cores · 6000G / 節點
適用於大型資料集與
記憶體密集型應用



共享高速儲存 /home · /work

提供高速、共同的資料儲存空間，連結登入節點、資料傳輸節點與所有運算資源

Nano4 其他 AI 資源 (參考)

晶創26 提供多元 GPU 運算資源
供不同研究需求使用



H200 GPU 叢集
適用於多樣化 AI 運算應用



GB200 NVL72
新世代大規模 GPU 平台
(參考)

更強大的 AI 計算能量
支援多元研究可能

分析數據
運行模擬
推動生醫研究



計算 × 生命 × 健康
開創生醫新視野

高效運算 • 精準醫療 • 更健康的未來

§1 前端連線資訊：同一個主機名稱，兩個 Port

節點	主機名稱	Port	用途	連線工具
登入節點	nano4.nchc.org.tw	22	下指令、寫腳本、送 Slurm 作業、輕量除錯	Terminal、PowerShell、Antigravity / VS Code
資料傳輸節點 (DTN)	nano4.nchc.org.tw	2222	大檔案傳輸，直通 /home、/work	WinSCP、FileZilla、Cyberduck、scp、rsync

⚠ DTN 只能傳檔，不開放 Shell 指令；用 `ssh` 連 Port 2222 會被拒絕。

§1 三個角色：一個比喻

角色	比喻	你在這裡做什麼
登入節點	大樓接待大廳	連線、看檔案、寫腳本、小型測試
計算節點	工廠	真正的運算；不能直接走進去，要透過「工單」
Slurm	廠長	審查工單、分配機器、跑完把結果交給你

⚠ 登入節點不能跑重度運算：超過 5 分鐘、超過 4 核心或超過 8 GB 的工作，一律用 Slurm 送到計算節點。

§1 新手必知三大鐵律

1. 傳檔用 Port 2222：資料傳輸節點和登入節點同一個主機名稱 `nano4.nchc.org.tw`，但傳檔要指定 `-P 2222`
2. 登入節點是 x86_64：GB200 是 Arm 架構，不能在登入節點替 GB200 編譯或建環境（本課程不使用 GB200）
3. 只接受台灣境內 IP：在國外請先連 VPN 回台灣學術網路，或事先向 iService 申請

§2 前置準備：iService 計畫與 IDExpert 2FA

1. 註冊 iService 會員並加入計畫：本課程計畫代號 **GOV115088**
2. 建立主機帳號與密碼：在 iService 建立 Linux 主機帳號，密碼要有英文大小寫、數字與特殊符號
3. 綁定雙因子 (2FA) App：手機安裝 IDExpert，依 iService 雙因子認證設定手冊掃描 QR Code 完成綁定

這三步是課前作業；還沒完成的同學請舉手，助教會協助你。

§3 用什麼軟體登入 Nano4 ?

每台電腦都已經內建 SSH，不需要另外安裝軟體：

你的電腦	打開哪個軟體	怎麼打開
Windows 10 / 11	PowerShell (或 Windows Terminal)	開始選單搜尋「PowerShell」
macOS	終端機 (Terminal)	Cmd + 空白鍵 搜尋「終端機」
Linux	Terminal	Ctrl + Alt + T

打開後，先確認 SSH 可以用：

```
ssh -V
```

✅ 出現 `OpenSSH_...` 版本號就可以用

💡 Windows 出現「找不到 ssh」：到「設定 → 系統 → 選用功能」安裝 **OpenSSH 用戶端**

§3-A SSH 登入 Nano4

在**自己電腦**的終端機 (Windows 請開 PowerShell) 輸入：

```
ssh your_account@nano4.nchc.org.tw
```

- 第一次連線會問是否信任主機金鑰 → 輸入 **yes**
- 接著會出現 2FA 選單 (下一頁)
- 最後輸入主機密碼：**畫面不會顯示任何字元**，這是正常的

§3-B 三種 2FA 方式

Login method (1: Mobile APP OTP, 2: Mobile APP PUSH, 3: Email OTP):

選項	方式	怎麼做	推薦
2	手機推播	IDExpert App 跳出授權請求 → 按「同意」	★★★★★
1	手機 OTP	打開 IDExpert App → 左下角「OTP」→ 輸入 6 位數字	★★★★
3	Email OTP	到註冊信箱收驗證碼	備援

💡 推播沒收到？手動打開 IDExpert App，請求通常會馬上跳出；還是沒有就按 **Ctrl+C**，重新登入並改選 **1**。

§3-C 取得課程教材

登入成功後，把教材下載到家目錄：

```
cd "$HOME"
if [[ -d Nano4-Docs/.git ]]; then
    git -C Nano4-Docs pull --ff-only
else
    git clone https://github.com/gemini960114/Nano4-Docs.git
fi
cd "$HOME/Nano4-Docs"
git status --short
```

之後重新開啟終端機，都先回到教材目錄：

```
cd "$HOME/Nano4-Docs"
```

§4 讓登入更快：設定 SSH Config

在自己電腦編輯 `~/.ssh/config` (Windows : `C:\Users\<你>\.ssh\config`) :

```
# 晶創26 (Nano4) 登入節點 (x86_64 架構)
Host nano4
    HostName nano4.nchc.org.tw
    User your_account
    Port 22
    ServerAliveInterval 60
    ServerAliveCountMax 3

# 晶創26 (Nano4) 資料傳輸節點 (DTN 專用 · Port 2222)
Host nano4-dtn
    HostName nano4.nchc.org.tw
    User your_account
    Port 2222
    ServerAliveInterval 60
    ServerAliveCountMax 3
```

之後只要輸入 `ssh nano4` 就能登入。

每一條新連線還是要輸入密碼與 OTP。第 02 章的 `ssh-proxy` 會解決這個問題。

§5 資料傳輸節點：大檔案一律走 Port 2222

⚠ 不要用登入節點 (Port 22) 傳數十 GB 的資料：登入節點頻寬有限、多人共用

圖形化工具 (WinSCP / FileZilla / Cyberduck)

欄位	填入
檔案協定	SFTP
主機名稱	nano4.nchc.org.tw
連接埠	2222 (預設 22 會連到登入節點)
帳號 / 密碼	iService 主機帳號與密碼，登入時完成 IDExpert 2FA

§5 命令列傳檔：sftp / scp / rsync

```
# 互動式 SFTP ( put 上傳、get 下載、quit 離開 )  
sftp -P 2222 your_account@nano4.nchc.org.tw
```

```
# 上傳檔案至 /work 高速暫存工作區  
scp -P 2222 dataset.tar.gz your_account@nano4.nchc.org.tw:/work/your_account/
```

```
# 下載分析結果到本地端  
scp -P 2222 -r your_account@nano4.nchc.org.tw:/work/your_account/output/ ./local_results/
```

```
# rsync：支援斷點續傳與進度顯示，最推薦  
rsync -avzP -e "ssh -p 2222" ./my_dataset/ your_account@nano4.nchc.org.tw:/work/your_account/my_dataset/
```

 scp、sftp 用大寫 -P 2222；rsync 要寫成 -e "ssh -p 2222"

§6 從台灣杉三號 (T3) 搬遷資料 (參考)

只有原本使用台灣杉三號 (`t3-c4.nchc.org.tw`) 的學員需要：

- 在 **Nano4 登入節點** 執行，由 Nano4 主動連回 T3
- 先搬到 `/work/$USER` 的暫存目錄，**不要直接覆寫** Nano4 的 `$HOME`
- 先用 `-n` 預覽 (dry-run)，確認清單正確再正式同步

```
hfsquota
mkdir -p /work/$USER/t3-home-backup

# 只預覽，不會寫入資料；替換成實際 T3 帳號
rsync -avHSn --info=progress2 \
    <T3帳號>@t3-c4.nchc.org.tw:/home/<T3帳號>/ \
    /work/$USER/t3-home-backup/
```

 資料量大時在 `tmux` 內執行；完整步驟 A-D 見講義第 01 章 §6。

§7 登入後第一步：環境健檢

執行本章隨附的一鍵健檢腳本：

```
cd "$HOME/Nano4-Docs/01-nano4-ssh-and-2fa/scripts"  
./quick_healthcheck.sh
```

報告會依序列出：

1. 節點與系統資訊：主機名稱、作業系統、CPU 與記憶體
2. 計畫與 SU 錢包餘額 (`wallet`)
3. `/home` 與 `/work` 的容量與剩餘空間
4. Lmod、Apptainer、`uv` 是否可用
5. Slurm 佇列概況 (本課程 `ngs62g`)
6. 外網連通性 (NCBI、GitHub)

§7 三大儲存空間：什麼東西放哪裡？

路徑	放什麼	本課程配額	注意
<code>\$HOME</code> (<code>/home/帳號</code>)	程式碼、Git repo、設定檔、skill	約 100 GB	不要放大量小檔案或快取
<code>/work/帳號</code>	FASTQ、分析結果、容器與暫存檔	約 100 GB ~1.5 TB	沒有備份 ，重要結果要自己備份 國科會計畫提供 1.5TB 免費空間
<code>/project</code>	跨成員共享的計畫資料	需另外申請	由計畫主持人向國網簽約申請
<code>/tmp</code>	✗ 不要放任何資料	—	隨時會被清除

⚠ 是 `/work`，不是舊系統的 `/work1`。

✗ 沒有 `sudo`：軟體請用 `module`、`uv` 或 Apptainer 容器解決。

§7 查詢自己的空間：hfsquota

```
hfsquota
```

PATH	USED	HARD LIMIT	USAGE %	STATUS
/home/<帳號>	499,712 B	107,374,182,400 B	0	ACTIVE
/work/<帳號>	2,240,512 B	107,374,182,400 B	0	ACTIVE

✗ 不要用 `df -h`：它顯示的是整個叢集好幾 PB 的容量，不是你的配額。

📌 分析完成後，要長期保存的資料移到 GP1-4 大容量儲存服務，不要一直放在 `/work`。



練習 1：環境健檢（5 分鐘）

```
whoami      # 我是誰
hostname    # 我在哪台登入節點 ( 25a-lgn01 ~ 05 )
pwd          # 我在哪個資料夾
hfsquota    # 我的 /home 與 /work 配額
```

執行一鍵健檢腳本：

```
cd "$HOME/Nano4-Docs/01-nano4-ssh-and-2fa/scripts"
./quick_healthcheck.sh
```

請確認：

- `wallet` 那一段有列出 **GOV115088**
- 佇列概況顯示 `ngs62g` (本課程 GOV115088 · 固定 `-c 8 --mem=62G · 96h`)
- 外網測試出現  外網連線正常 (NCBI 連通)

練習項目

§2 前置準備：iService 計畫與 IDExpert 2FA

§3-A SSH 登入 Nano4

§3-B 三種 2FA 方式

§3-C 取得課程教材

§4 讓登入更快：設定 SSH Config

§7 登入後第一步：環境健檢

§7 查詢自己的空間：hfsquota



練習 2：用 Port 2222 雙向傳檔 (FileZilla)

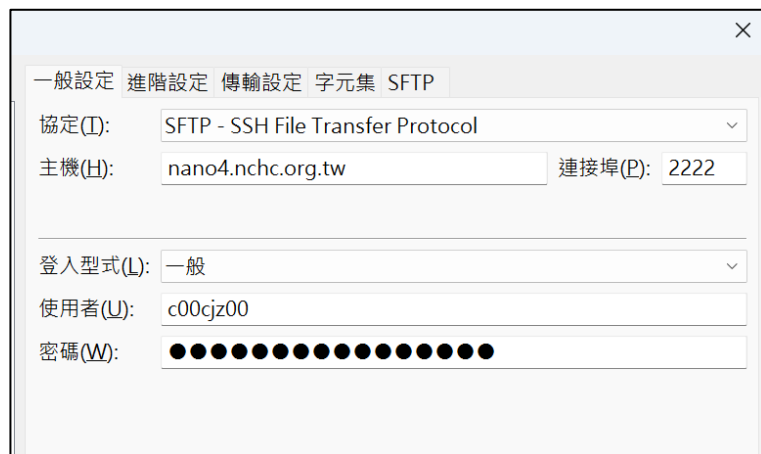
Download Filezilla

<https://filezilla-project.org/>

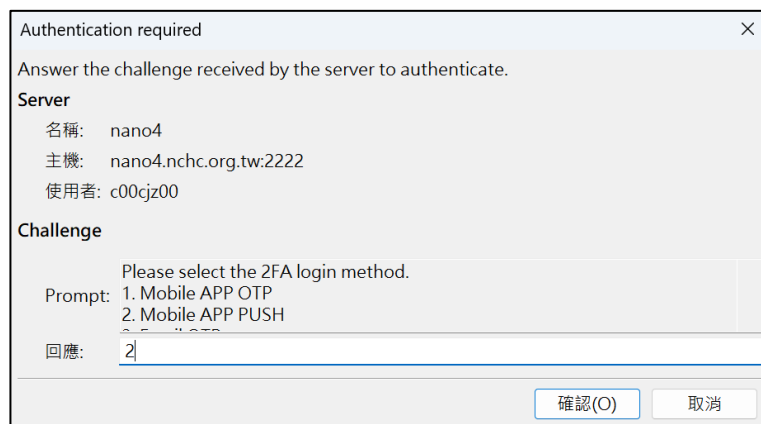
練習項目

§5 資料傳輸節點：大檔案一律走 Port 2222

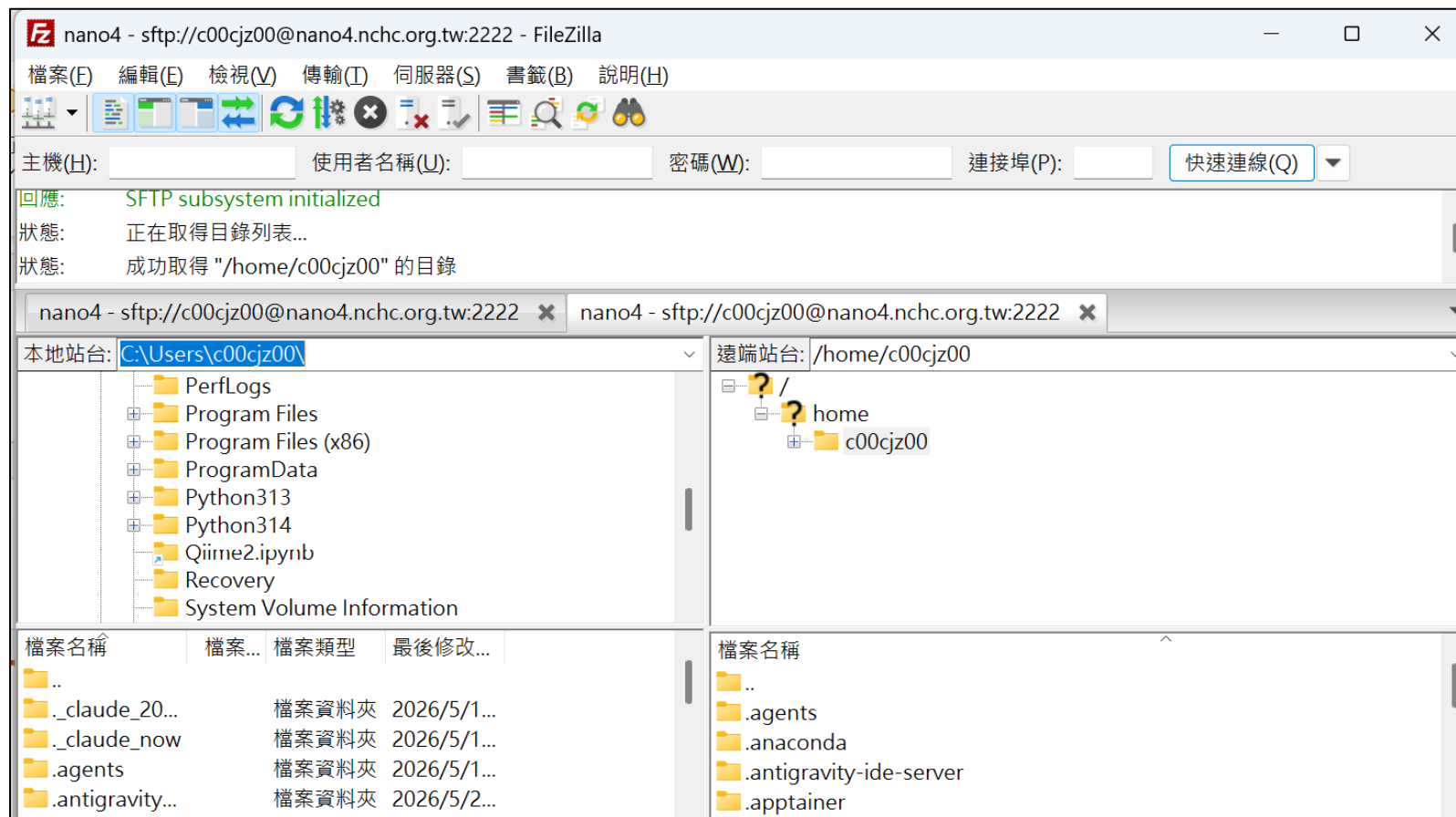
§5 命令列傳檔：Filezilla / sftp / scp / rsync



FileZilla Client Settings dialog box. The 'General' tab is selected. The 'Protocol' is set to 'SFTP - SSH File Transfer Protocol'. The 'Host' is 'nano4.nchc.org.tw' and the 'Port' is '2222'. The 'Login type' is 'Normal'. The 'Username' is 'c00cjz00' and the 'Password' is masked with dots.



Authentication required dialog box. It shows the challenge received by the server to authenticate. The 'Server' section shows 'Name: nano4', 'Host: nano4.nchc.org.tw:2222', and 'User: c00cjz00'. The 'Challenge' section shows the prompt 'Please select the 2FA login method.' with options '1. Mobile APP OTP' and '2. Mobile APP PUSH'. The response is '2'.



FileZilla Client interface showing the connection to nano4 - sftp://c00cjz00@nano4.nchc.org.tw:2222. The status bar shows 'SFTP subsystem initialized' and '正在取得目錄列表...'. The local site shows the file system structure of C:\Users\c00cjz00\, including folders like PerfLogs, Program Files, ProgramData, Python313, Python314, Qiime2.ipynb, Recovery, and System Volume Information. The remote site shows the file system structure of /home/c00cjz00, including folders like .agents, .anaconda, .antigravity-ide-server, and .apptainer.



練習 2：用 Port 2222 雙向傳檔（選做）



步驟 1、2、4 在自己電腦的終端機執行，步驟 3 在 Nano4 上執行

```
# 1. ( 自己電腦 ) 建立測試檔案
echo "Hello Nano4! This is my first file." > hello_nano4.txt

# 2. ( 自己電腦 ) 上傳到 Nano4 的 /work，記得加 -P 2222
scp -P 2222 hello_nano4.txt your_account@nano4.nchc.org.tw:/work/your_account/

# 3. ( Nano4 ) 確認檔案已上傳
cat /work/$USER/hello_nano4.txt

# 4. ( 自己電腦 ) 把檔案下載回來
scp -P 2222 your_account@nano4.nchc.org.tw:/work/your_account/hello_nano4.txt ./downloaded_test.txt
```



用 Port 22 傳檔會出現 Connection refused

練習項目

\$5 資料傳輸節點：大檔案一律走 Port 2222

\$5 命令列傳檔：Filezilla / sftp / scp / rsync

§8 軟體從哪裡來？module

超算上不能 `sudo apt install`，官方已經把常用軟體裝好，用 `module`（簡寫 `ml`）載入：

指令	簡寫	用途
<code>module avail biology/FastQC</code>	<code>ml avail biology/FastQC</code>	查詢有哪些版本
<code>module spider <名稱></code>	<code>ml spider <名稱></code>	全域搜尋工具
<code>module load 名稱/版本</code>	<code>ml 名稱/版本</code>	載入
<code>module list</code>	<code>ml</code>	列出目前載入的模組
<code>module purge</code>	<code>ml purge</code>	全部清空

📌 生醫工具都在 `biology/` 底下，例如 `biology/FastQC`、`biology/Nextflow`、`biology/SAMtools`。



練習 3：載入生醫模組（5 分鐘）

練習項目

S8 軟體從哪裡來？module

1. 清空環境，確認工具還不存在

```
module purge
```

```
command -v fastqc || echo "fastqc: 尚未載入"
```

2. 載入本課程會用到的三個模組

```
module load biology/JDK/26.0.1 biology/FastQC/0.11.9 biology/MultiQC
```

3. 確認版本

```
fastqc --version          # FastQC v0.11.9
```

```
multiqc --version         # multiqc, version 1.35
```

```
java -version             # java version "26.0.1"
```

```
module list
```

4. 再次清空，工具又不見了

```
module purge
```

```
command -v fastqc || echo "fastqc: 已隨 module purge 移除"
```

§8 為什麼 FastQC 一定要搭配 JDK ?

- 計算節點沒有系統 Java
- 只載入 `biology/FastQC`、沒載入 `biology/JDK` 時：
 - FastQC 找不到 Java，不會產生任何報告
 - 但指令仍可能顯示成功 ← 最容易被騙的地方
- AI Agent 也常漏掉這一點，今天最後的示範請注意看

這也是為什麼 Slurm 腳本第一行要 `module purge`：先清空，再明確載入需要的東西。

§9 Apptainer 容器（第三堂實作）

沒有 `sudo` 又需要複雜的軟體環境時，用 **Apptainer**（前身為 Singularity）執行 Docker 映像檔：

-  在容器內仍是一般使用者，不會破壞主機安全
-  整個環境打包成一個 `.sif` 檔，容易保存與搬移
- Nano4 已預載 Apptainer 1.4.3（`apptainer` 與 `singularity` 都可以用）

快取目錄一定要放 `/work`，避免塞爆 `$HOME`：

```
export APPTAINER_CACHEDIR="/work/${USER}/.apptainer_cache"  
export SINGULARITY_CACHEDIR="/work/${USER}/.singularity_cache"
```

```
apptainer exec --nv docker://pytorch/pytorch:latest python -c "import torch; print('CUDA 可用:',  
torch.cuda.is_available())"
```

 映像檔要在 Slurm 作業中拉取，不要在登入節點拉大型映像；第三堂 Lab 9 會實作。

§10 uv : 取代 conda 的 Python 套件管理

- 一個 conda 環境常有 5 ~ 10 萬個小檔案，很容易用光 Inode 配額
- uv 速度比 pip / conda 快 10 ~ 100 倍，並用硬連結避免重複檔案
- 單一執行檔，不需要管理者權限

找不到 uv 時，用使用者權限安裝到 ~/.local/bin (不需要 sudo) :

```
export PATH="${HOME}/.local/bin:${PATH}"
if ! command -v uv >/dev/null 2>&1; then
    curl -LsSf https://astral.sh/uv/install.sh | sh
    export PATH="${HOME}/.local/bin:${PATH}"
fi
uv --version
```



練習 4：用 uv 建立 Python 環境（選做）

```
# 1. 快取放 /work，避免塞爆 $HOME
export UV_CACHE_DIR="/work/${USER}/.uv_cache"

# 2. 在 /work 建立虛擬環境
uv venv /work/${USER}/lab_env

# 3. 安裝套件
uv pip install --python /work/${USER}/lab_env/bin/python rich requests

# 4. 啟動環境並測試，最後離開
source /work/${USER}/lab_env/bin/activate
python -c "from rich import print; print('[bold green] 🎉 Python 虛擬環境啟動成功！[/bold green]')"
deactivate
```



虛擬環境一律建在 `/work/${USER}`，不要放在 `$HOME`

練習項目

\$10 uv：取代 conda 的 Python 套件管理

§11 為什麼需要 Slurm ?

[個人電腦思維] 雙擊程式 → 馬上在本機執行 → 關掉視窗就中斷

[超級電腦思維] 在登入節點寫腳本 → `sbatch` 交給 Slurm
→ 計算節點在背景執行 → 你可以關機回家
→ 跑完結果寫進日誌檔

 今天只看 Slurm 怎麼用，第二堂再學腳本每一行的意思。

§11 ngs62g 規格：官方固定搭配

佇列	核心 -c	記憶體 --mem	最長時間	本課程
ngs62g	8	62 GB	4 天	✓ GOV115088 唯一可用
ngstest、ngs8g ~ ngs32g、ngs125g...	—	—	—	✗ 只開放 MST109178 等計畫

- 國網規定每個佇列都要用固定的「核心 × 記憶體」申請
- 送到其他 ngs* 佇列會被拒絕
- 計費以核心小時計算
- H200 / GB200 GPU 佇列只供參考，本課程不使用



練習 5：第一次提交 Slurm 批次作業 (1/2)

複製 CPU 範本到自己的工作區，並檢視內容：

練習項目

\$11 為什麼需要 Slurm？

\$11 ngs62g 規格：官方固定搭配

```
cp "$HOME/Nano4-Docs/01-nano4-ssh-and-2fa/scripts/sample_first_cpu_job.slurm" /work/$USER/my_first_cpu_job.slurm
cd /work/$USER
cat my_first_cpu_job.slurm
```

關鍵參數	意思
#SBATCH --account=GOV115088	本課程計畫代號
#SBATCH --partition=ngs62g	生醫專屬 CPU 佇列
#SBATCH --cpus-per-task=8	8 顆 CPU 核心
#SBATCH --mem=62G	62 GB 記憶體 (ngs62g 必須搭配 -c 8 --mem=62G)

✗ 純 CPU 佇列不要加 `--gres=gpu:1`



練習 5：送出、追蹤與檢查效能（2/2）

```
sbatch my_first_cpu_job.slurm      # → Submitted batch job 422203
queue --me                          # PD = 排隊中、R = 執行中
cat first_cpu_job-*.out             # 跑完後看日誌
seff <JOB_ID>                       # CPU 與記憶體使用效率
```

- `seff` 會印出 CPU 利用率（CPU Utilized）與記憶體利用率（Memory Efficiency）
- `<JOB_ID>` 就是 Submitted batch job 後面的數字

💡 作業幾秒就跑完，`queue` 看不到是正常的。

練習項目

§11 為什麼需要 Slurm？

§11 ngs62g 規格：官方固定搭配



練習 6：salloc 互動式計算節點（時間夠再做）

```
salloc --account=GOV115088 --partition=ngs62g --nodes=1 \  
--cpus-per-task=8 --mem=62G -t 00:30:00 srun --pty /bin/bash
```

申請成功後，會直接進入計算節點：

練習項目

§11 為什麼需要 Slurm？

§11 ngs62g 規格：官方固定搭配

```
salloc: Granted job allocation 421820  
salloc: Nodes 25a-cpn01 are ready for job  
[user@25a-cpn01 ~]$
```

hostname	# 應顯示 25a-cpn*；若顯示 25a-lgn* 表示仍在登入節點
nproc	# 應顯示 8（申請的核心數）
exit	# ⚠ 用完一定要離開，釋放資源（停止計費）

離開後執行 `squeue --me`，清單中沒有這個作業才代表資源已釋放。

§12 六個入門練習回顧

練習	內容	接在哪一節	本堂課
1	登入後的環境健檢	§7	必做
2	用 Port 2222 雙向傳檔	§5	選做（在自己電腦操作）
3	載入生醫模組、體驗 <code>module purge</code>	§8	必做
4	用 <code>uv</code> 建立 Python 環境	§10	選做（會寫 Python 的學員）
5	第一次提交 Slurm 批次作業	§11	時間夠再做（第二堂會完整練習）
6	<code>salloc</code> 互動式計算節點	§11	時間夠再做

 沒做完的練習改為課後自學，講義第 01 章 §12 有完整步驟。

§13 第 01 章常見問題

狀況	解法
ssh 卡住，Connection timed out	是否在國外？單位防火牆擋 Port 22？改用手機熱點試試
SFTP / WinSCP 出現 Connection refused	連到 Port 22 了，傳檔要用 Port 2222
推播一直沒收到	手動打開 IDExpert App；或 Ctrl+C 重來並改選 1 OTP
Disk quota exceeded	hfsquota 檢查；大檔案移到 /work/\$USER
sudo 被拒絕	正常。改用 module、uv 或容器
在登入節點跑程式被 Killed	超過 5 分鐘的重度運算會被系統清除，改用 sbatch 或 salloc
GB200 出現 Exec format error (參考)	登入節點是 x86_64，要進 gb200-dev 節點重建環境

第 02 章

Antigravity Remote-SSH 與三個 AI Agent

第 02 章：8 個小節與 6 個練習

§	主題	本堂課
1	為什麼要用 Antigravity Remote-SSH	講解
2	連上 Nano4：步驟 A-E (含 ssh-proxy) →  練習 1、2	實作
3	裝上三個 AI Agent：安裝擴充套件、登入	實作
4	三個 Agent 怎麼讀到規則與 Skills	講解
5	AGENTS.md 治理守則 →  練習 3、4、5	實作
6	動手練習 1~6 (練習 6 是結尾示範)	實作 / 示範
7	多登入節點與 Agent session 清理	參考
8	常見問題 FAQ	參考

§1 為什麼要用 Antigravity Remote-SSH ?

```
[ 你的筆電 ]
└─ Antigravity ( 本機介面、快捷鍵 )
    │ 加密 SSH 連線 ( 第 01 章設定的 Host nano4 )
    ▼
[ Nano4 登入節點 25a-lgn01~05 ]
├─ Antigravity Server ( 背景執行 )
├─ 三個 AI Agent : 內建 Agent、Codex、Claude Code
├─ /work/$USER 高速工作區
└─ Slurm 指令 ( sbatch、squeue、sacct )
```

- 目的：把 AI Agent 帶到超算上，讓它直接讀檔、寫腳本、送作業
- 滑鼠操作檔案總管，不用記 `vim`；`Ctrl +`、開啟遠端終端機
- 用 VS Code 也可以，步驟相同，只是沒有內建 Agent

§2 步驟 A：安裝 Antigravity

1. 到 <https://antigravity.google/> 下載安裝，用 Google 帳號登入
2. `Ctrl + Shift + X` 開啟擴充套件，確認已有 **Remote - SSH** (已內建就略過)
3. 首次連線時，Antigravity 會在 Nano4 的 `~/.antigravity-ide-server` 安裝伺服器端

用 VS Code 的同學

1. 到 <https://code.visualstudio.com/> 下載安裝
2. 搜尋 **Remote - SSH** (Microsoft 發行) → Install

§2 步驟 B-C：連上 Nano4

1. 點左下角  圖示 → **Connect to Host...**
2. 選 **nano4** (讀取第 01 章設定的 `~/.ssh/config`)
3. 平台選 **Linux**
4. 上方輸入框出現 2FA 選單 → 輸入 **2** → 手機按同意 → 輸入密碼
5. 左下角顯示 **SSH: nano4** 就成功了

💡 卡在「Waiting for 2FA」？提示可能縮在視窗頂部，或在「Output → Remote - SSH」分頁裡。

§2 步驟 D + 練習 1：開啟遠端工作區

1. 檔案總管 → **Open Folder** → 輸入 `/work/your_account`
2. 右鍵新增檔案 `my_test.py`，寫入：

```
print("Hello from VS Code Remote on Nano4!")
```

3. `Ctrl + S` 存檔，按 `Ctrl + `` 開啟終端機：

```
python my_test.py
```

👉 注意：剛才 **Open Folder** 時，是不是又要輸入一次密碼和 OTP？

練習項目

§1 為什麼要用 Antigravity Remote-SSH？

§2 步驟 A：安裝 Antigravity

§2 步驟 B-C：連上 Nano4

§2 步驟 E 問題：為什麼要一直認證？

Nano4 的**每一條新 SSH 連線**都要密碼 + OTP。

Antigravity 在這些時候都會開新連線：

- 第一次連線
- Open Folder 選目錄
- 切換資料夾、重新載入視窗

→ 一堂課可能要認證**好幾次** 😞

§2 步驟 E 解法：ssh-proxy 只認證一次

Antigravity / VS Code / 終端機
| 連 nano4-proxy (不需 OTP)
▼
127.0.0.1:2222 ← ssh-proxy (在你的電腦上執行，視窗保持開著)
| 已完成密碼 + OTP 的連線
▼
nano4.nchc.org.tw:22

- ssh-proxy 先幫你完成一次認證，並保持連線
- 之後所有連線都走這條已認證的通道
- 額外好處：所有連線固定在同一台登入節點

 <https://github.com/gemini960114/ssh-proxy>

§2 ssh-proxy 設定 1：下載

到 Releases 頁面 (<https://github.com/gemini960114/ssh-proxy/releases/latest>)
下載 (免安裝 Python)：

你的電腦	下載檔案
Windows 10 / 11	ssh-proxy-windows-x64.exe
macOS Apple Silicon (M1–M4)	ssh-proxy-macos-arm64
Linux x64	ssh-proxy-linux-x64

Intel Mac 沒有預先編譯的檔案，需依 ssh-proxy README 用 `uv` 從原始碼執行。

§2 ssh-proxy 設定 2：加入 nano4-proxy

在自己電腦的 `~/.ssh/config` 再加一段（保留原本的 `Host nano4`）：

```
Host nano4-proxy
  HostName 127.0.0.1
  Port 2222
  User your_account
  # 這兩行只能用在 127.0.0.1 這個別名，絕對不要加到 nano4
  StrictHostKeyChecking no
  UserKnownHostsFile /dev/null
  LogLevel ERROR
  ServerAliveInterval 30
  ServerAliveCountMax 3
```

⚠ 記得把 `your_account` 改成自己的帳號。

§2 ssh-proxy 設定 3：啟動並認證一次

在自己電腦的終端機執行，並讓這個視窗保持開著：

```
# Windows (PowerShell)
.\ssh-proxy-windows-x64.exe nano4 --max-lifetime 10h

# macOS Apple Silicon ( 第一次需加執行權限並解除下載隔離 )
chmod +x ssh-proxy-macos-arm64
xattr -d com.apple.quarantine ssh-proxy-macos-arm64
./ssh-proxy-macos-arm64 nano4 --max-lifetime 10h
```

- 第一次會顯示 Nano4 的主機金鑰指紋 → 輸入完整的 `yes`
- 依提示輸入密碼與 OTP
- `--max-lifetime 10h`：預設 8 小時會自動停止，加上這個撐過一整天



練習 2：確認只需認證一次（10 分鐘）

另開一個本機終端機，連續連線兩次：

```
ssh nano4-proxy hostname  
ssh nano4-proxy 'echo $USER; hfsquota'
```

練習項目

\$2 ssh-proxy 設定 1：下載

\$2 ssh-proxy 設定 2：加入 nano4-proxy

\$2 ssh-proxy 設定 3：啟動並認證一次

✓ 兩次都直接顯示結果，不再要求密碼或 OTP

接著在 Antigravity：

1. Connect to Host... → 選 **nano4-proxy**（不是 nano4）

2. Open Folder 開啟 /work/<帳號>

3. 再切換到 /home/<帳號>/Nano4-Docs

✓ 切換資料夾時不再出現 OTP

§2 ssh-proxy 使用注意

狀況	說明
自動停止	沒有任何連線 60 分鐘 ，或總執行 8 小時 後會停止（可用 <code>--max-lifetime</code> 延長）
Connection refused	proxy 已停止 → 重新啟動（再認證一次）並重新連線
Address already in use	已經有另一個 proxy 在用 2222 埠，先關掉它
REMOTE HOST IDENTIFICATION HAS CHANGED	proxy 會在送出密碼前停止；先向國網中心確認， 不要 直接刪除舊金鑰

 **只在自己的電腦上使用：**proxy 執行期間，同一台電腦的其他程式也能透過它進入你的帳號。離開座位或使用共用電腦時，請在 proxy 視窗按 `Ctrl+C`。

§3 三個 AI Agent，自由切換

Agent	怎麼取得	需要的帳號
Antigravity 內建 Agent	內建，不需安裝	Google 帳號（已登入）
Codex	擴充套件 <code>openai.chatgpt</code>	ChatGPT 帳號
Claude Code	擴充套件 <code>anthropic.claude-code</code>	Claude 帳號或 API key

- 同一個問題，請三個 Agent 各回答一次，互相比對
- 你不必背指令：說出想做的分析，由 Agent 寫腳本、送作業、解讀結果

只有 Antigravity 內建 Agent 也能完成所有練習；有帳號的同學再加裝另外兩個來比較。

§3-A 安裝擴充套件：裝在遠端

連上 `nano4-proxy` 後，按 `Ctrl + `` 開啟終端機：

```
cd "$HOME/Nano4-Docs/02-vscode-and-ai-tools/scripts"  
bash install_vscode_extensions.sh
```

- 腳本會安裝 Codex、Claude Code，以及 Python、Jupyter 套件
- 也可以在 Extensions 面板搜尋，點 **Install in SSH: nano4-proxy**

⚠ Agent 要裝在遠端 (SSH)，才能讀取 Nano4 上的檔案、在 Nano4 上執行指令。

§3-A 腳本會安裝哪些套件？

套件類別	識別碼 (Extension ID)	核心功能
Codex	<code>openai.chatgpt</code>	OpenAI 的 AI Agent
Claude Code	<code>anthropic.claude-code</code>	Anthropic 的 AI Agent
Python 核心環境	<code>ms-python.python</code>	Python 語法高亮、虛擬環境自動偵測
Python 除錯器	<code>ms-python.debugpy</code>	斷點除錯、單步執行與變數檢視
Jupyter 互動運算	<code>ms-toolsai.jupyter</code>	執行 <code>.ipynb</code> Notebook 的運算引擎
Jupyter 渲染器	<code>ms-toolsai.jupyter-renderers</code>	支援 Plotly 互動圖表與 DataFrame 表格檢視

 Antigravity 內建 Agent 不需要安裝；Codex 與 Claude Code 需要學員自己的帳號。

§3-B 登入各個 Agent

1. Codex：點左側活動列的 Codex 圖示 → Sign in with ChatGPT
2. Claude Code：點左側活動列的 Claude 圖示 → 登入 Claude 帳號（或填 API key）
3. Antigravity 內建 Agent：打開 Agent 面板即可使用

⚠ 三個 Agent 都在登入節點執行指令：運算一定要讓 Agent 用 `sbatch` 送到計算節點。

§3-B Agent 要執行指令時，會先問你

Codex、Claude Code 在執行指令或寫入檔案前，會跳出確認視窗：

1. 先看它要跑什麼，不要直接按「允許」
2. 看到 `sbatch` → 檢查帳號、佇列與 `-c 8 --mem=62G`
3. 看到在登入節點直接執行 `fastqc`、`multiqc` 等運算 → **拒絕**，請它改用 `sbatch`
4. 看不懂的指令 → 先問 Agent：「這個指令是做什麼的？」

👉 你負責確認，AI 負責執行。

§4 Agent 從哪裡讀到 Nano4 的規則與 Skills ?

Agent	讀取的規則檔	工作資料夾的 Skills	個人的 Skills
Antigravity 內建 Agent	<code>.agents/rules/</code> (教材的 <code>nano4.md</code> 會引用 <code>AGENTS.md</code>)	<code>.agents/skills/</code>	<code>~/.gemini/config/skills/</code>
Codex	<code>AGENTS.md</code>	<code>.agents/skills/</code>	<code>~/.agents/skills/</code>
Claude Code	<code>CLAUDE.md</code> (教材的 <code>CLAUDE.md</code> 匯入 <code>AGENTS.md</code>)	<code>.claude/skills/</code>	<code>~/.claude/skills/</code>

- 用 Open Folder 開啟 `$HOME/Nano4-Docs` , 三個 Agent 都會讀到規則
- 開啟其他資料夾時讀不到 → 在對話開頭說「請先閱讀 `AGENTS.md`」
- Skills 第二堂才會用到：學員會做出自己的 skill `my-nano4-slurm`

§5 AI 不知道超算的規矩

AI 助手預設不知道超級電腦是多人共用的，可能會建議：

- ❌ `sudo apt install ...`
- ❌ 把 50 GB 模型下載到 `$HOME`
- ❌ 在登入節點直接跑大型運算
- ❌ 為了「省資源」把 `--mem` 調小

👉 解法：在 repo 根目錄放一份 `AGENTS.md`，告訴 AI 這台機器的規則。

§5 AGENTS.md : 給 AI 的 Nano4 守則

1. 禁止 `sudo` : 軟體改用 `module`、`uv` 或 Apptainer 容器
2. 大資料放 `/work/$USER` : 不塞爆 `$HOME` , 不寫 `/tmp`
3. 登入節點不跑重度運算 : 一律封裝成 Slurm 作業
4. Slurm 標準 : `module purge` 放第一行、日誌用 `%x-%j.out`、不反覆查詢 `squeue`
5. 本課程設定 : GOV115088 只能用 `ngs62g` , 固定 `-c 8 --mem=62G`
6. 替使用者操作 : 先用白話說明計畫、等你同意再送出 , 跑完用白話解讀結果



練習 3：裝上三個 Agent 並打招呼（10 分鐘）

1. Open Folder 開啟 `/home/<帳號>/Nano4-Docs`
2. 執行 `install_vscode_extensions.sh`，登入你有帳號的 Agent
3. 對每個 Agent 輸入同一句話：

請先閱讀 `AGENTS.md`。用三句話告訴我：我現在在哪一台電腦上？這門課我能用哪個計畫和佇列？每個作業要申請多少核心和記憶體？

✓ 三個 Agent 都能說出 `25a-1gn0X`、`G0V115088`、`ngs62g` 與 `-c 8 --mem=62G`

練習項目

- §3 三個 AI Agent，自由切換
- §3-A 安裝擴充套件：裝在遠端
- §3-A 腳本會安裝哪些套件？
- §3-B 登入各個 Agent
- §3-B Agent 要執行指令時，會先問你
- §4 Agent 從哪裡讀到 Nano4 的規則與 Skills？
- §5 AI 不知道超算的規矩
- §5 AGENTS.md：給 AI 的 Nano4 守則



練習 4：請 Agent 解釋 Slurm 範本（5 分鐘）

選一個 Agent 輸入下列 prompt（時間夠再換另一個 Agent 比較）：

請先閱讀 AGENTS.md，再解釋 03-slurm-syntax-and-job-management/templates/standard_cpu_job.slurm 每一個 #SBATCH 參數的用途，但先不要修改檔案。

比較它們的回答：

- 是否都說出 **GOV115088** 只能使用 **ngs62g**？
 - 有沒有 Agent 建議把 **--mem** 調小？
- 這是**錯誤建議**：國網規定 **ngs62g** 必須固定 **-c 8 --mem=62G**
- 哪個 Agent 的說明最容易懂？哪個最精確？



練習 5 A：用自然語言請 AI 查資源 (1/2)

三個 prompt 由淺入深，都只查詢、不送作業，每一個產出一份報告：

Prompt 1：查詢 Slurm Partition

請使用 `sinfo` 與 `scontrol show partition`，列出這台 HPC 上的 Slurm Partition：名稱、節點數、狀態、每個節點的 CPU 核心數與記憶體、最長執行時間 (MaxTime)。
只查詢、不要送出任何作業。結果整理成表格，存到 `/work/$USER/day1_ai_query/partition.md`。

Prompt 2：查詢我的計畫

請執行 `wallet`，列出我名下所有計畫代碼 (Project ID)、計畫名稱與剩餘額度 (SU)，整理成表格存到 `/work/$USER/day1_ai_query/project.md`。



練習 5 A：確認 GOV115088 能用哪些佇列 (2/2)

Prompt 3：驗證計畫權限

請確認計畫 GOV115088 可以使用 ngs8g、ngs16g、ngs32g、ngs62g、ngs125g 之中的哪些 Partition：

1. 用 `sacctmgr -nP show assoc user="$USER" account="gov115088"` 確認我有這個計畫的 Slurm 授權 (Slurm 裡的帳號名稱是小寫)。
2. 用 `scontrol show partition` 檢查這五個 Partition 的 AllowAccounts。
3. 對能用的 Partition，用 `sacctmgr show qos p_<Partition 名稱>` 查出每個作業可申請的核心與記憶體上限。

判斷依據必須來自指令的輸出，不要只引用 AGENTS.md。用表格列出每個 Partition 能不能用、依據與規格。

只查詢、不要送出任何作業。結果存到 `/work/$USER/day1_ai_query/permission.md`。

✓ `permission.md` 的結論應該是只有 `ngs62g` 可用，每個作業 `cpu=8,mem=62G`



練習 5 B：人工驗證 AI 的結論 (1/2)

AI 的回答只是建議，最後要由你自己驗證：

```
scontrol show partition ngs62g | grep -o "AllowAccounts=[^ ]*"
sacctmgr -nP show qos p_ngs62g format=Name,MaxTRESPerJob
```

```
AllowAccounts=mst109178,ent109430,gov115088
p_ngs62g|cpu=8,mem=62G
```



比對 `permission.md` 和自己查到的結果是否一致。



練習 5 B：驗證練習 4 的範本 (2/2)

```
cd "$HOME/Nano4-Docs"  
sinfo -p ngs62g  
bash -n 03-slurm-syntax-and-job-management/templates/standard_cpu_job.slurm  
sbatch --test-only --account=GOV115088 \  
    03-slurm-syntax-and-job-management/templates/standard_cpu_job.slurm
```

預期結果：

```
( bash -n 沒有任何輸出 = 語法正確 )  
sbatch: Job 426753 to start at 2026-09-23T20:19:09 using 8 processors on nodes 25a-cpn01 in  
partition ngs62g
```

⚠ `--test-only` 只檢查帳號與分區，不檢查官方規格 (`-c 8 --mem=62G`)，這部分要自己對照。

練習 6 A：手動送出質控作業

4 個定序樣本 (FASTQ , 各 1000 reads) → FastQC 逐一檢查 → MultiQC 彙整成一份報告

```
cd /work/$USER
sbatch "$HOME/Nano4-Docs/02-vscode-and-ai-tools/templates/day1_fastqc_multiqc.slurm"
sacct -X -o JobID,JobName,State,Elapsed      # 跑完後 State 為 COMPLETED
tail -3 day1_qc-*.out
```

日誌最後一行：

✅ 完成！MultiQC 報告：[/work/<帳號>/day1_qc/multiqc_out/multiqc_report.html](#)

💡 作業約 10 秒就跑完。腳本每一行的意思，第二堂會學到。



練習 6 B：一句自然語言，請 Agent 做同一件事

在工作資料夾為 `Nano4-Docs` 的 Antigravity，對任一個 Agent 輸入：

我有 4 個定序樣本 (FASTQ)，放在 `04-ai-assisted-bio-pipeline/demo_data/fastq_raw/`。
請幫我檢查這些樣本的定序品質：用 FastQC 逐一檢查，再用 MultiQC 彙整成一份報告，結果放在 `/work/$USER/day1_qc_ai/`。
請依照 `AGENTS.md` 把工作送到計算節點執行。送出前先用三到五句話告訴我你的計畫，等我同意再送出；跑完後用白話告訴我這 4 個樣本的品質如何。



你沒有打任何指令，只說了想完成的分析

打開 MultiQC 報告，看一眼成果

報告在遠端 Nano4 上，兩種打開方式：

方式 A：下載到筆電

檔案總管找到 `/work/<帳號>/day1_qc/multiqc_out/multiqc_report.html` → 按右鍵 → **Download...** → 在筆電瀏覽器開啟

方式 B：連接埠轉送

```
cd "$HOME/Nano4-Docs/04-ai-assisted-bio-pipeline/scripts"  
bash view_multiqc_report.sh /work/$USER/day1_qc/multiqc_out
```

- 依畫面提示在瀏覽器打開 `http://localhost:<埠號>/multiqc_report.html`
- 看完執行 `tmux kill-session -t svc-multiqc-report` 關閉預覽

📌 先看一眼：4 個樣本的品質分數都在綠色區。怎麼讀懂整份報告，第二堂第 04 章會教。

練習 6：比較兩種送法

檢查項目	手動腳本	AI 產生的腳本
--account=GOV115088 、 --partition=ngs62g	✓	?
--cpus-per-task=8 、 --mem=62G	✓	?
module purge 後載入 biology/JDK/26.0.1 biology/FastQC/0.11.9 biology/MultiQC	✓	?
用 sbatch 送到計算節點，不在登入節點直接跑	✓	?
送出後沒有反覆查詢 squeue	✓	?
用白話解讀品質結果	—	?

⚠ Agent 做錯時，用說的告訴它違反了 AGENTS.md 哪一條，讓它修正後重來。

§7 多登入節點與 Agent session 清理

- 連線可能被分到 25a-1gn01 ~ 25a-1gn05 不同主機；舊主機上的 Agent 沒有結束時，看起來會像 session 卡住
- 💡 使用 ssh-proxy 時，所有連線固定在同一台登入節點，比較不會遇到

```
cd "$HOME/Nano4-Docs"
chmod +x 02-vscode-and-ai-tools/scripts/kill.sh \
        02-vscode-and-ai-tools/scripts/kill_login.sh

# 只清理目前這台登入節點：會先列出匹配程序，再要求確認
bash 02-vscode-and-ai-tools/scripts/kill.sh

# 清理五台登入節點上的 stale agent ( 不會再詢問確認，會直接執行 )
bash 02-vscode-and-ai-tools/scripts/kill_login.sh
```

⚠️ 請從一般 SSH 終端機 (`ssh nano4`) 執行，不要在 Antigravity 整合終端機內執行，否則會立即斷線。

§8 第 02 章常見問題

狀況	解法
卡在「Waiting for 2FA...」	看視窗頂部輸入框，或 Output → Remote - SSH 分頁
找不到 <code>/work</code>	File → Open Folder，手動輸入 <code>/work/你的帳號/</code>
連 <code>nano4-proxy</code> 出現 <code>Connection refused</code>	proxy 視窗被關掉或已逾時，重新啟動
找不到 Codex 或 Claude Code	確認左下角是 <code>SSH: nano4-proxy</code> ，再裝到遠端
Agent 不知道 <code>G0V115088</code> 、 <code>ngs62g</code>	工作資料夾要開 <code>\$HOME/Nano4-Docs</code> ，開新對話再問
Jupyter 選不到 Kernel	遠端要裝 <code>ms-toolsai.jupyter</code> 、 <code>ms-python.python</code> ，環境要裝 <code>ipykernel</code>

今天最重要的一件事

AI Agent 替你寫指令、送作業、解讀結果。

你負責確認它的計畫與結果是否正確。

- 確認計畫：帳號、佇列、`-c 8 --mem=62G`、資料放哪裡
- 確認結果：作業真的 `COMPLETED`、報告真的產生
- 發現錯誤：用說的糾正它

👉 第二堂：把這些「糾正 AI」的經驗，存成你自己的 skill

今天學到了什麼

主題	重點
連線	<code>ssh nano4</code> + 2FA ; <code>ssh-proxy</code> 讓 Antigravity 只認證一次
儲存	程式碼放 <code>\$HOME</code> 、資料放 <code>/work</code> 、不放 <code>/tmp</code> ; 用 <code>hfsquota</code> 查配額
軟體	<code>module load</code> ; FastQC 要搭配 <code>biology/JDK</code>
Slurm	<code>GOV115088</code> → <code>ngs62g</code> → 固定 <code>-c 8 --mem=62G</code>
AI Agent	三個 Agent 裝在遠端 ; <code>AGENTS.md</code> 讓它們守規則 ; 用自然語言派送作業，自己驗證結果

下課前檢核

- 1. 能透過 `nano4-proxy` 用 Antigravity 開啟 `/work/<帳號>`，而且不再輸入 OTP
- 2. 至少一個 Agent 能說出 `GOV115088`、`ngs62g` 與 `-c 8 --mem=62G`
- 3. 執行 `sacct` 查得到練習 6 的作業：

```
sacct -X -o JobID,JobName,State,Elapsed
```

- 4.  離開前在 proxy 視窗按 `Ctrl+C`（尤其是共用電腦）

課後自學（選做）

章節	內容
第 01 章 練習 2	用 Port 2222 雙向傳檔 (<code>scp -P 2222</code>)
第 01 章 練習 4	用 <code>uv</code> 在 <code>/work</code> 建立 Python 環境
第 01 章 練習 6	用 <code>salloc</code> 進入互動式計算節點
第 01 章 §6	從台灣杉三號 (T3) 搬遷資料
第 02 章 練習 6 B	換另一個 Agent 再做一次，比較兩個 Agent 的腳本

下一堂預告

第二堂：Slurm 與我的 skill

- 開場暖身：在沒有規則檔的資料夾請 AI 送作業，看看它會漏掉什麼
- 第 03 章：親手送出標準、陣列、相依作業，用 `seff` 檢查資源
- 用自然語言請 Agent 多輪完成同樣的工作，並糾正它的錯誤
- 把經驗存成你自己的 skill：`my-nano4-slurm`
- 第 04 章：用自己的 skill 做 FASTQ 質控，和 Agent 一起讀懂報告

📖 講義：<https://gemini960114.github.io/Nano4-Docs/>

謝謝大家！🙌